

Ai-Service-Desk

Technical Documentation

Date: July 24, 2026, **Version:** 1.0

Revision History

Version	Date	Author	Description
1.0	24-Jul-2026	Midhun	Initial draft

Table of Contents

Technical Documentation.....	1
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Intended Audience.....	1
1.4 References.....	1
1.5 Acronyms and Definitions.....	1
2. System Overview.....	2
2.1 High-Level Architecture.....	2
2.2 System flow - Key Components.....	3
2.3 User Roles and Access Levels.....	3
3. Functional Requirements.....	3
3.1 Module/Feature Name.....	3
3.2 Inputs/Outputs.....	4
3.3 Business Rules.....	4
3.4 Error Handling.....	4
4. Non-Functional Requirements.....	4
4.1 Performance.....	4
4.2 Security.....	5
4.3 Availability.....	5
4.4 Maintainability.....	5
5. Data Model / Database Design.....	6
5.1 Entity-Relationship Diagram.....	6
5.2 Tables and Fields.....	6
5.3 Indexes and Keys.....	6
5.4 Sample Data.....	7
6. Integration Points.....	7
6.1 External Systems.....	7
6.2 APIs Used.....	7
6.3 Data Flow Diagram.....	8
6.4 Authentication/Authorization.....	8

7. Test Strategy.....	8
7.1 Test Scenarios.....	8
7.2 Expected Results.....	9
7.3 Traceability to Requirements.....	9
8. Deployment & Configuration.....	9
8.1 Environments.....	9
8.2 Setup Steps.....	9
8.3 Configuration Files and Parameters.....	9
9. Appendix.....	10
Glossary.....	10
Screenshots.....	10
Code Snippets.....	11
References.....	11

1. Introduction

1.1 Purpose

This document describes the technical design and implementation of the AI Service Desk, a FastAPI-based REST API for managing support tickets with AI-powered summarization. It aims to give developers, reviewers, and QA engineers a single reference for the system's architecture, CRUD APIs, data model, validation rules, middleware, testing strategy, and deployment process.

1.2 Scope

In scope: the FastAPI application layer, Pydantic request/response validation, async SQLAlchemy ORM integration with PostgreSQL, Alembic migrations, custom middleware, the AWS Bedrock AI summarization integration, unit testing with pytest, and Docker-based deployment.

Out of scope: frontend/UI implementation (the API is consumed via Swagger UI or external clients), production-grade authentication/authorization (not yet implemented), and infrastructure-as-code for cloud hosting.

1.3 Intended Audience

Audience	Purpose
Backend Developers	Understand the application architecture, APIs, business logic, and database interactions.
QA / Test Engineers	Perform unit and integration testing to ensure application quality.
DevOps Engineers	Deploy, configure, and maintain the application in different environments.
Technical Reviewers / Mentors	Review the system design, implementation, and overall technical quality.

1.4 References

FastAPI official documentation, SQLAlchemy 2.0 asynchronous ORM documentation, Alembic migration documentation, AWS Bedrock user guide, and the pytest documentation. The project's requirements.txt and pyproject.toml enumerate exact dependency versions.

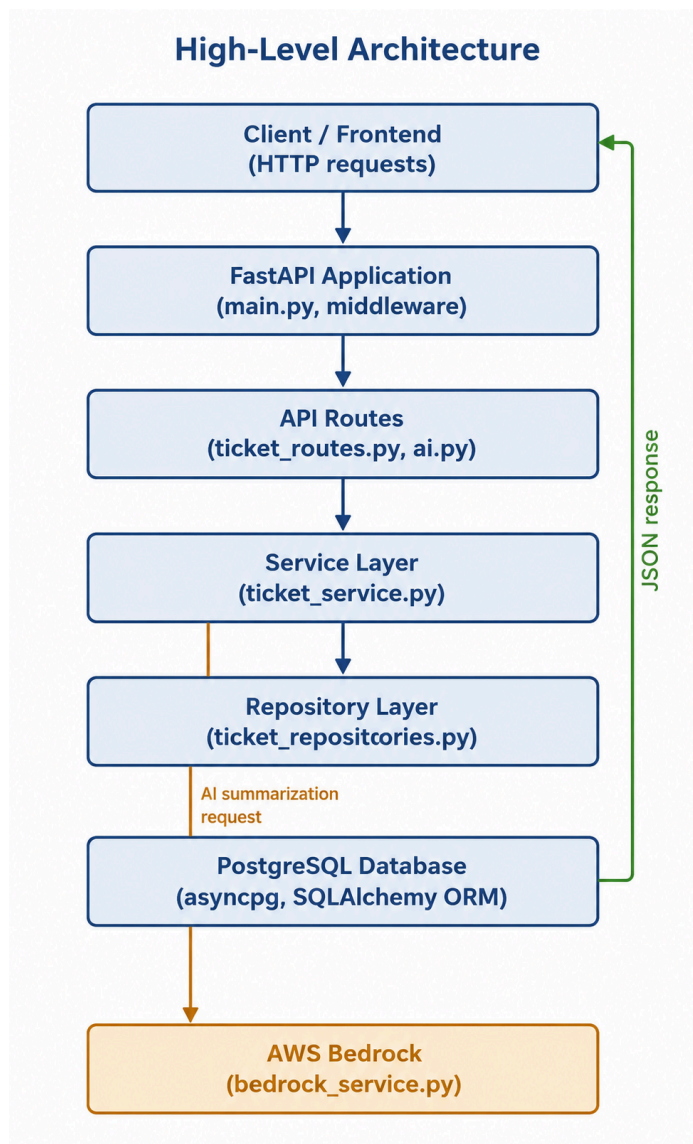
1.5 Acronyms and Definitions

API - Application Programming Interface
CRUD - Create, Read, Update, Delete
ORM - Object-Relational Mapping
ASGI - Asynchronous Server Gateway Interface
UUID - Universally Unique Identifier
CORS - Cross-Origin Resource Sharing
DFD - Data Flow Diagram
AWS - Amazon Web Services

TDD - Test-Driven Development
JSON - JavaScript Object Notation

2. System Overview

2.1 High-Level Architecture



The AI Service Desk follows a layered, async-first architecture on top of FastAPI:
Client -> API Routes (ticket_routes.py, ai.py) -> Service Layer (ticket_service.py, bedrock_service.py) -> Repository Layer (ticket_repositories.py) -> SQLAlchemy Async ORM -> PostgreSQL (asyncpg driver)

The AI summarization path branches from the service layer to bedrock_service.py, which calls the AWS Bedrock converse API using Jinja2-rendered prompt templates. Uvicorn serves the ASGI

application, and Docker packages the app for deployment. Alembic manages schema migrations independently of application startup.

2.2 System flow - Key Components

main.py - FastAPI app setup, middleware registration, exception handlers
core/config.py - Pydantic Settings for DB URL, AWS credentials, app metadata
core/database.py - AsyncEngine and AsyncSessionLocal configuration
core/deps.py - get_db() dependency injected into routes
core/exceptions.py - TicketNotFoundError custom exception
models/ticket.py - SQLAlchemy ORM model with TicketStatus and TicketPriority enums
schemas/ticket_schema.py - Pydantic Create/Update/Response schemas
services/ticket_service.py - CRUD business logic
services/bedrock_service.py - AWS Bedrock AI integration
services/prompt_templates.py - Jinja2 prompt templates for summarization
repositories/ticket_repositories.py - SQLAlchemy query layer
api/ticket_routes.py and api/ai.py - route definitions
A request enters through a route, is validated by a Pydantic schema, passed to the service layer for business logic, then to the repository layer for the actual async database query, with the async session injected via get_db().

2.3 User Roles and Access Levels

User Role	Permissions
Requester	Create tickets and view their own ticket status (POST /tickets, GET /tickets/{id}).
Support Agent	Update ticket status/priority and trigger AI summarization (PUT /tickets/{id}, POST /ai/summarize); identified via assignee_email.
Administrator	Full CRUD access, including ticket deletion.

3. Functional Requirements

3.1 Module/Feature Name

Ticket CRUD Module (ticket_routes.py / ticket_service.py / ticket_repositories.py) - creates, lists, retrieves, updates, and deletes support tickets, with filtering by status and priority on the list endpoint.

Pydantic Validation Module (schemas/ticket_schema.py) - defines TicketCreate, TicketUpdate, and TicketResponse schemas that validate field types, lengths, and enum values before data reaches the service layer.

Database Connectivity and ORM Integration (core/database.py, models/ticket.py) - async SQLAlchemy engine and sessions map the Ticket entity to the PostgreSQL tickets table, with Alembic handling schema evolution.

Middleware Module (main.py) - CORS middleware and a response-time middleware that

attaches an X-Response-Time header to every response.

AI Summarization Module (services/bedrock_service.py, api/ai.py) - sends a ticket description to AWS Bedrock via the converse API using a Jinja2 prompt template and returns a generated summary.

3.2 Inputs/Outputs

Endpoint	Input	Output
POST /tickets	TicketCreate JSON (title, description, priority, assignee_email)	TicketResponse with generated id, status, timestamps
GET /tickets	Optional status / priority query filters	List of TicketResponse
GET /tickets/{ticket_id}	UUID path parameter	Single TicketResponse, or 404 if not found
PUT /tickets/{ticket_id}	TicketUpdate JSON (partial fields)	Updated TicketResponse
DELETE /tickets/{ticket_id}	UUID path parameter	204 No Content
POST /ai/summarize	Ticket description text	AI-generated summary string from AWS Bedrock

3.3 Business Rules

title is required, max length 200 characters; description is optional, max length 500 characters; priority must be one of low, medium, high, critical (default medium); status must be one of open, in_progress, resolved, closed (default open); assignee_email is optional and validated as a well-formed email by Pydantic; id, created_at, and updated_at are server-generated and not accepted on create.

3.4 Error Handling

Pydantic validation failures are automatically converted by FastAPI into HTTP 422 responses with field-level error detail. Lookups for a non-existent ticket raise a custom TicketNotFoundError, which is caught by a dedicated exception handler registered in main.py and converted into an HTTP 404 JSONResponse. Unhandled exceptions surface as HTTP 500 responses during development with DEBUG=True enabled for detailed tracebacks.

4. Non-Functional Requirements

4.1 Performance

Measured with cProfile (profile_test.py) and Locust:

Create Ticket - approximately 200-250 ms end-to-end, approximately 0.2 s at the database layer (INSERT + SELECT)

List Tickets - approximately 20-50 ms (single SELECT)

Get Single Ticket - approximately 0.005 s (SELECT by id)

Update Ticket - approximately 0.015 s (UPDATE + SELECT)

Delete Ticket - approximately 0.01 s (DELETE)

Liveness check (/health) - under 1 ms

Readiness check (/ready) - approximately 50-100 ms, including a DB connectivity check

Load characteristics beyond these single-user profiling numbers are exercised with locust.py.

4.2 Security

AWS credentials (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY) and the database URL are supplied via environment variables and never hard-coded. CORS middleware currently allows all origins, which is acceptable for development but should be restricted before production use.

Token-based authentication (SECRET_KEY, ACCESS_TOKEN_EXPIRE_MINUTES) is configured but not yet enforced on routes - this is a known gap tracked for a future revision. Pydantic validation at the schema layer also serves as a first line of defense against malformed or malicious input.

4.3 Availability

The current setup runs as a single Uvicorn/FastAPI instance against a single PostgreSQL instance with no configured failover or redundancy - suitable for development and demonstration.

Docker containerization allows the application to be scaled horizontally behind a load balancer, and SQLAlchemy's async connection pooling helps the service tolerate transient database latency, but high-availability database replication and multi-instance orchestration are not yet implemented.

4.4 Maintainability

The layered architecture (routes / services / repositories / models) isolates concerns so business logic, data access, and API contracts can be changed independently. Alembic provides reversible, auto-generatable migrations for schema changes. The pytest suite and cProfile/Pyinstrument profiling scripts make regressions and performance issues easy to catch during development. Pydantic schemas centralize validation rules, reducing the risk of inconsistent checks across endpoints.

5. Data Model / Database Design

5.1 Entity-Relationship Diagram

Entity-Relationship Diagram

tickets	
id	UUID (PK)
title	VARCHAR(200)
description	VARCHAR(500)
priority	ENUM
status	ENUM
assignee_email	VARCHAR(254)
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

The current data model consists of a single entity, Ticket, with no foreign-key relationships to other tables. Conceptually: Ticket (id PK) stands alone, with assignee_email stored as a plain field rather than a foreign key to a separate Users/Agents table - a normalization opportunity noted for future iterations.

5.2 Tables and Fields

Field Name	Data Type	Description
id	UUID (PK)	Unique identifier for each ticket.
title	VARCHAR(200)	Short title describing the issue. Required.
description	VARCHAR(500)	Detailed description of the support request. Optional.
priority	ENUM	low, medium, high, critical - default medium.
status	ENUM	open, in_progress, resolved, closed - default open.
assignee_email	VARCHAR(254)	Email of the assigned support agent. Optional.
created_at	TIMESTAMPTZ	Server-generated creation timestamp.
updated_at	TIMESTAMPTZ	Server-generated last-updated timestamp.

5.3 Indexes and Keys

id is the primary key (UUID, indexed by default via the primary key constraint). No additional secondary indexes are currently defined; status and assignee_email are candidates for future

indexes given they are the primary filter and lookup fields on the list endpoint. There are no foreign keys in the current single-entity model.

5.4 Sample Data

id	title	priority	status	assignee_email	created_at
3fa85f64-5717-4...afa6	Unable to reset password	high	open	agent@example.com	2026-07-20 09:15
7c1e2a4b-88d3-4...b21f	Dashboard shows 500 error	high	in_progress	agent2@example.com	2026-07-21 11:40
a5f9d3c7-11ab-4...44e2	Feature request: dark mode	low	resolved	agent@example.com	2026-07-18 16:05

6. Integration Points

6.1 External Systems

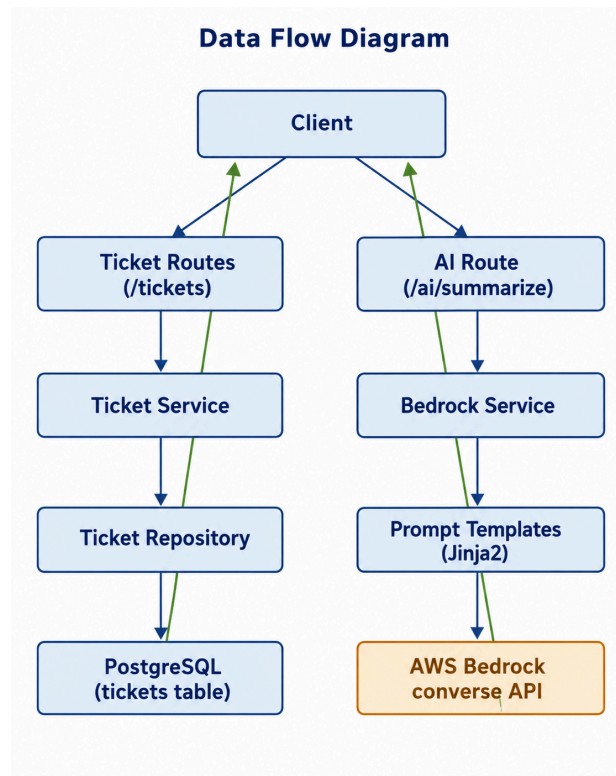
AWS Bedrock - provides the large language model used to generate ticket summaries via the converse API.

PostgreSQL - the system of record for all ticket data, accessed asynchronously through asyncpg.

6.2 APIs Used

API	Type	Purpose
GET /health	Exposed	Liveness probe - confirms the application process is running.
GET /ready	Exposed	Readiness probe - checks the database connection is available.
POST /tickets	Exposed	Create a new support ticket.
GET /tickets	Exposed	Retrieve all tickets, filterable by status/priority.
GET /tickets/{id}	Exposed	Retrieve a specific ticket by UUID.
PUT /tickets/{id}	Exposed	Update an existing ticket.
DELETE /tickets/{id}	Exposed	Delete a ticket.
POST /ai/summarize	Exposed	Generate an AI-powered summary of a ticket description.
AWS Bedrock converse API	Consumed	Generates the AI summary using the configured foundation model.

6.3 Data Flow Diagram



Ticket CRUD flow: Client -> ticket_routes.py -> ticket_service.py -> ticket_repositories.py -> PostgreSQL (async session) -> response mapped back through TicketResponse.

AI summarization flow: Client -> api/ai.py -> bedrock_service.py (renders a prompt via prompt_templates.py) -> AWS Bedrock converse API -> summarized text returned to the client. Every response additionally passes through the response-time middleware, which appends the X-Response-Time header before returning to the client.

6.4 Authentication/Authorization

No login mechanism or role-based access control is enforced in the current version - all endpoints are publicly reachable, which is appropriate only for local development. SECRET_KEY and ACCESS_TOKEN_EXPIRE_MINUTES are already defined in configuration as a foundation for adding JWT-based authentication and role checks in a future release.

7. Test Strategy

7.1 Test Scenarios

test_ticket_create.py - creating a ticket with valid data, and with missing/invalid fields

test_ticket_get.py - retrieving a single ticket, listing tickets, filtering by status/priority, and retrieving a non-existent ticket

test_ticket_update.py - partial and full updates to an existing ticket, and updating a non-existent ticket

test_ticket_delete.py - deleting an existing ticket and deleting a non-existent ticket

7.2 Expected Results

Valid create requests return HTTP 201 with a fully populated TicketResponse. Invalid payloads return HTTP 422 with field-level validation errors. Reads return the expected ticket(s) or an empty list when filters match nothing. Updates persist only the fields supplied and return the refreshed record. Deletes return HTTP 204, and subsequent reads of a deleted ticket return HTTP 404. Any operation on a non-existent UUID returns HTTP 404 via TicketNotFoundError.

7.3 Traceability to Requirements

test_ticket_create.py traces to sections 3.1/3.2/3.3 (CRUD module, inputs/outputs, business rules). test_ticket_get.py and test_ticket_update.py trace to sections 3.1/3.2 and 4.1 (performance). test_ticket_delete.py traces to sections 3.1 and 3.4 (error handling). profile_test.py and locust.py trace to section 4.1 (performance) directly.

8. Deployment & Configuration

8.1 Environments

Development - local machine with PostgreSQL running via Docker on localhost:5432, DEBUG=True, used for day-to-day feature work.

Testing - pytest run locally or in CI against a test database, using pytest-asyncio in auto mode.

Production - containerized deployment via the provided Dockerfile, expected to run with DEBUG=False and restricted CORS origins.

8.2 Setup Steps

1. Create and activate a Python 3.11+ virtual environment.
2. Install dependencies: pip install -r requirements.txt
3. Set AWS credentials as environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY).
4. Run Alembic migrations: alembic upgrade head
5. Start the server: python main.py or uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
6. For Docker: docker build -t ai-service-desk:latest . then docker run -p 8000:8000 with the required environment variables.

8.3 Configuration Files and Parameters

Configuration is supplied via a .env file loaded through Pydantic Settings, including DATABASE_URL, DEBUG, APP_NAME, APP_VERSION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION, BEDROCK_MODEL_ID, SECRET_KEY, and

ACCESS_TOKEN_EXPIRE_MINUTES. Each deployment environment should supply its own values for the database connection and AWS credentials rather than reusing development defaults.

9. Appendix

Glossary

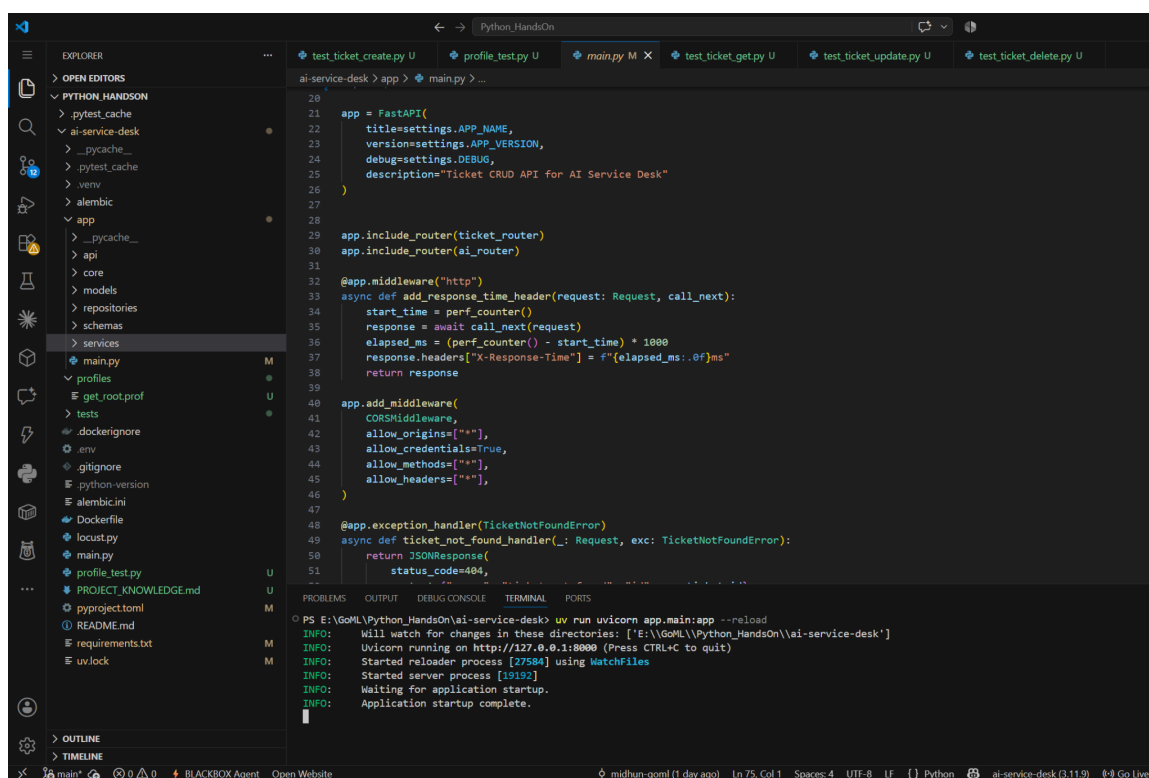
Ticket - a support request tracked by the system, with a title, description, priority, and status.

CRUD - the four core data operations: Create, Read, Update, Delete.

Async session - a SQLAlchemy database session that performs non-blocking I/O using asyncpg.

Summarization - the process of condensing a ticket description into a shorter form using an AWS Bedrock language model.

Screenshots



```

20
21 app = FastAPI(
22     title=settings.APP_NAME,
23     version=settings.APP_VERSION,
24     debug=settings.DEBUG,
25     description="Ticket CRUD API for AI Service Desk"
26 )
27
28 app.include_router(ticket_router)
29 app.include_router(ai_router)
30
31 @app.middleware("http")
32 async def add_response_time_header(request: Request, call_next):
33     start_time = perf_counter()
34     response = await call_next(request)
35     elapsed_ms = (perf_counter() - start_time) * 1000
36     response.headers["X-Response-Time"] = f"{elapsed_ms:.0f}ms"
37     return response
38
39 app.add_middleware(
40     CORSMiddleware,
41     allow_origins=["*"],
42     allow_credentials=True,
43     allow_methods=["*"],
44     allow_headers=["*"],
45 )
46
47 @app.exception_handler(TicketNotFoundError)
48 async def ticket_not_found_handler(_: Request, exc: TicketNotFoundError):
49     return JSONResponse(
50         status_code=404,
51     )

```

```

PS E:\GoML\Python_HandsOn\ai-service-desk> uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['E:\GoML\Python_HandsOn\ai-service-desk']
INFO: Uvicorn running on https://127.0.0.1:8080 (Press CTRL+C to quit)
INFO: Started reloader process [27584] using WatchFiles
INFO: Started server process [19192]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

Ticket CRUD API 0.1.0 OAS 3.1

[OpenAPI JSON](#)

Ticket CRUD API for AI Service Desk

Tickets		^
POST	/tickets Create Ticket	▼
GET	/tickets Get All Tickets	▼
GET	/tickets/{ticket_id} Get Ticket By Id	▼
PUT	/tickets/{ticket_id} Update Ticket	▼
DELETE	/tickets/{ticket_id} Delete Ticket	▼
AI		^
POST	/ai/summarize Summarize Ticket	▼
Root		^
GET	/ Home	▼
System		^
GET	/health Health	▼
GET	/ready Ready	▼

Code Snippet

Illustrative ticket creation endpoint pattern:

```
@router.post("/tickets", response_model=TicketResponse, status_code=201)
async def create_ticket(payload: TicketCreate, db: AsyncSession = Depends(get_db)):
    return await ticket_service.create_ticket(db, payload)
```

Illustrative Pydantic schema pattern:

```
class TicketCreate(BaseModel):
    title: str = Field(max_length=200)
    description: str | None = Field(default=None, max_length=500)
    priority: TicketPriority = TicketPriority.medium
    assignee_email: EmailStr | None = None
```

References

FastAPI documentation — <https://fastapi.tiangolo.com>

SQLAlchemy 2.0 documentation — <https://docs.sqlalchemy.org>

Alembic documentation — <https://alembic.sqlalchemy.org>

Pydantic v2 documentation — <https://docs.pydantic.dev>

Amazon Bedrock documentation — <https://docs.aws.amazon.com/bedrock>

boto3 (AWS SDK for Python) documentation — <https://boto3.amazonaws.com>

Docker & Docker Compose documentation — <https://docs.docker.com>

pytest documentation — <https://docs.pytest.org>

Locust documentation — <https://docs.locust.io>